RoboSim / RoboAnalyzer: Project Overview (Part 2 of 2)

This document represents the second half of the RoboSim project exploration, focusing on the **UI/UX Architecture**, **Data Pipelines**, and the **3D Rendering Engine**.

---

4. Frontend Architecture (PyQt6)

The frontend is built entirely using **PyQt6**, heavily leveraging Qt's Signal/Slot mechanism to pass data safely across application threads. The UI uses a custom "Kinetic Obsidian" dark theme, applying CSS-like stylesheets directly to Qt widgets.

4.1. Application Layout (`main_window.py`)

The `MainWindow` class orchestrates the entire application. It divides the screen into: * **Left Dock:** The `ArmCanvas` (3D rendering). * **Bottom Dock:** The `ReplayControlBar` and `DataPanel` (graphs). * **Right Dock:** A scrollable area containing modular "Panels" that control various aspects of the simulation.

4.2. Control Panels

The UI is heavily modularized into distinct Python files under `frontend/panels/`: * **`connection_panel.py`:** Handles switching between Live Hardware, Simulator, and CSV Replay modes. It emits signals that tell `MainWindow` which background worker thread to spawn. * **`kinematic_chain_panel.py`:** The "Robot Builder". It provides a UI to add/remove custom DH joints. When the user modifies a joint, this panel reconstructs the `KinematicChain` and pushes it to the `ArmCanvas`. * **`trajectory_panel.py`:** Provides an interactive 3D XYZ coordinate picker (spinboxes and sliders). It directly hooks into the backend's Inverse Kinematics solver. It also features a "Batch Automation" system to process CSV waypoints sequentially. * **`replay_bar.py`:** Transport controls (Play, Pause, Step, Scrubber). It connects directly to the `ReplayController`'s state machine to control frame emission.

---

5. Data Pipelines & Multi-Threading

A core challenge in robotics UI is plotting data in real-time without freezing the interface. RoboSim solves this using a strict Producer-Consumer threading model.

5.1. QThread Workers

Instead of running heavy loops on the main UI thread, data generation happens in background threads: * `SimWorker` and `ReplayWorker` subclass `QThread`. They run infinite loops inside `run()`, crunching numbers or reading from files. * They emit a PyQt Signal: `sample_received(dict)`. * **Thread Safety:** PyQt automatically marshals this signal across thread boundaries, delivering the `dict` safely to the main thread's event queue.

5.2. The Rendering Delegate Loop

When `MainWindow` receives a frame from a worker, it triggers `_render_delegate(frame)`. This single function acts as the central nervous system: 1. **Extract Data:** It reads the joint angles (q_1, q_2, q_3, etc.) from the telemetry frame. 2. **Forward Kinematics:** It calls `KinematicChain.forward_kinematics(angles)` to determine exactly where every robot link currently is in 3D space. 3. **Update UI:** It passes these physical 3D coordinates to the `ArmCanvas` for drawing, and updates the 2D scatter plots in the `EEMapPanel`.

---

6. 3D Rendering & Optimization (Matplotlib)

Rendering 3D graphics in Python is notoriously slow if done incorrectly. RoboSim uses `matplotlib.pyplot` embedded inside PyQt via `FigureCanvasQTAgg`.

6.1. `arm_canvas.py`

This is the heart of the visualizer. It manages an `Axes3D` object. * **Initialization (`_init_empty_plot`):** It sets up the grid, limits, axes, and camera angles exactly once. * **Line/Scatter Artists:** Instead of clearing the plot and redrawing from scratch (which takes ~100ms per frame), RoboSim stores references to the `Line3D` and `Path3DCollection` objects.

6.2. Blitting & Artist Updates

When a new frame arrives, `update_plot(positions)` is called: 1. It iterates through the N joint coordinates provided by the Forward Kinematics engine. 2. It simply updates the `.set_data(X, Y)` and `.set_3d_properties(Z)` on the *existing* line objects. 3. This dramatically improves framerates, allowing smooth 60fps animations.

6.3. The End-Effector Path Trace

RoboSim tracks the robot's historical trajectory dynamically. * In `arm_canvas.py`, `_ee_trace_points` stores the last known XYZ coordinates of the End-Effector. * A persistent red line artist (`_ee_trace_line`) connects these points. * The code uses `try/except` blocks gracefully around

Matplotlib's `remove()` functions to prevent the UI from crashing if the user switches robot models while a trace is actively rendering.

---

7. Advanced Modules

7.1. Live Data Analysis (`data_panel.py`)

This panel tracks the real-time values of X, Y, Z and the individual joint angles. * It implements a **Ring Buffer** (e.g., keeping only the last 100 data points). * It dynamically updates `PyQtGraph` (a much faster plotting library than Matplotlib for simple 2D line charts).

7.2. 2D Scatter Map (`ee_map_panel.py`)

This provides a top-down ("drone view") scatter plot of the End-Effector's path. * It utilizes PySide/PyQt graphics scenes to plot individual dots representing the X-Y position of the tool tip, highly useful for visualizing the reach envelope of Scara or Planar robots.

Summary

RoboSim marries rigorous mathematical modeling (Denavit-Hartenberg matrices, Jacobians) with modern desktop software architecture (Thread-safe workers, asynchronous rendering loops, UI state machines). It is heavily optimized to process complex kinematics without blocking the user experience.